By default, a logout performed against an application protected by Spring Security will not affect Keycloak. Fortunately, Spring Security provides an implementation of the "OpenID Connect RP-Initiated Logout" specification, which defines how a logout request should be propagated from an OAuth2 Client (the Relying Party) to the Authorization Server. You'll see how to configure it for Edge Service soon.

NOTE The OpenID Connect specification includes a few different scenarios for session management and logout. If you'd like to learn more, I recommend you check the official documents for OIDC Session Management (https://openid.net/specs/openid-connect-session-1_0.xhtml), OIDC Front-Channel Logout (https://openid.net/specs/openid-connect-frontchannel-1_0.xhtml), OIDC Back-Channel Logout (https://openid.net/specs/openid-connect-backchannel-1_0.xhtml), and OIDC RP-Initiated Logout (https://openid.net/specs/openid-connect-rpinitiated-1_0.xhtml).

Spring Security supports logging out by sending a `POST` request to the `/logout` endpoint implemented and exposed by the framework by default. We want to enable the RP-Initiated Logout scenario so that when a user logs out of the application, they are also logged out of the Authorization Server. Spring Security has full support for this scenario and provides an `OidcClientInitiatedServerLogoutSuccessHandler` object you can use to configure how to propagate a logout request to Keycloak.

Let's say the RP-Initiated Logout feature is enabled. In that case, after a user has been successfully logged out of Spring Security, Edge Service will send a logout request to Keycloak through the browser (using a redirect). Next you'll probably want the user to be redirected back to the application after the logout operation has been performed on the Authorization Server as well.

You can configure where the user should be redirected after a logout with the `setPostLogoutRedirectUri()` method, which is exposed by the `OidcClientInitiatedServerLogoutSuccessHandler`